ชื่อ-นามสกุล _____ รหัสนักศึกษา _____

เฉลยชุดฝึกกลางภาคและเกณฑ์ตรวจ

CPE334 Software Engineering ภาคการศึกษาที่ 1/2569

วิธีใช้: อ่านคำตอบตัวอย่างก่อน แล้วใช้ **เกณฑ์ตรวจ** ตรวจว่าตอบครบหรือไม่ ไม่ต้องคัดเกณฑ์ลงคำตอบ ข้อ 1 ใช้รูป “เป็น... เพราะ...” พร้อมวิธีตรวจและสิ่งที่ต้องยืนยัน ข้อ 2.1 แยกครบ 8 องค์ประกอบ ส่วน 2.2 เขียนเป็นย่อหน้าเดียวได้ รับคำตอบอื่นที่ตรงโจทย์และมีเหตุผล ไม่หักความผิดต้นเหตุเดียวซ้ำ

แยกที่มา: ป้าย “คำอธิบายเสริม” ใช้กับข้อเสนอหรือตัวอย่างที่เติมเอง ไม่ใช่ข้อมูลที่ผู้สอนยืนยัน เลขหน้าอ้างอิงนับจากหน้าแรกของ PDF; ชื่อไฟล์เต็มอยู่ท้ายเฉลย

ส่วน	ข้อ	คะแนน
1 ความต้องการและความปลอดภัย	1-4	25
2 การวางแผนและดำเนินโครงการ	5-7	25
3 UML/UI และ Git	8-12	50
	รวม	100

คะแนนย่อยและเกณฑ์ตรวจเป็นของชุดฝึกนี้ ไม่ใช่ประกาศหรือเฉลยทางการของผู้สอน

ข้อ 1 วิเคราะห์ความต้องการ (8 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 5-6; Lecture 2 หน้า 14-26

คำย่อ: FR (Functional Requirement) บอกหน้าที่; NFR (Non-functional Requirement) บอกระดับคุณภาพ

1.1 ดู workshop ที่เปิดลงทะเบียน 2 คะแนน

เป็น FR เพราะให้นักศึกษาที่เข้าสู่ระบบดู workshop เพื่อเลือกสมัคร; คงข้อความได้หากตกลงคำว่า “เปิดรับ” แล้ว มิฉะนั้นต้องถามเพิ่ม; ทดสอบรายการเปิด/ปิด ต้องเห็นรายการเปิดครบและไม่เห็นรายการปิด

1.2 ลงทะเบียนรวดเร็ว 2 คะแนน

เป็น NFR ด้าน performance เพราะเน้นความเร็ว แต่ยังวัดไม่ได้; เสนอให้คำขออย่างน้อย 95% ตอบภายใน 2 วินาทีเมื่อมีผู้ใช้พร้อมกัน 100 คน (รอยืนยัน); ใช้ load test วัดตั้งแต่ส่งคำขอถึงได้ผล ต้องผ่านเกณฑ์นี้

1.3 ไม่เกิน 3 workshop ต่อภาค 2 คะแนน

เป็น business/domain rule เพราะจำกัดสิทธิ์ตามนโยบาย; คงเกณฑ์ 3 รายการต่อภาค แต่ยืนยันสถานะที่นับ; ทดสอบมี 2 เพิ่มเป็น 3 ต้องผ่าน ส่วนมี 3 ขอรายการที่ 4 ต้องถูกปฏิเสธและจำนวนยังเป็น 3

1.4 ยกเลิกและดูรายชื่อ 2 คะแนน

เป็น FR แต่ไม่ atomic เพราะรวมสองหน้าที่; แยกเป็นนักศึกษายกเลิกการสมัคร กับผู้ประสานงานดูรายชื่อ; ยืนยันสิทธิ์และเงื่อนไขแล้วทดสอบแต่ละบทบาท ต้องยกเลิกได้ตามสิทธิ์และรายชื่อตรงข้อมูล

คำอธิบายเสริม: ข้อ 1.2 เป็นตัวอย่างค่าที่เสนอ ไม่ใช่ค่าจากโจทย์ ต้องตกลงชุดข้อมูล เครือข่าย จุดวัดและกรณีผิดพลาดก่อนใช้รับงาน; ข้อ 1.3 ต้องถามว่าการยกเลิกนับรวมโควตาหรือไม่

เกณฑ์ตรวจต่อข้อย่อย 2 คะแนน: ประเภทพร้อมเหตุผล 0.25; จุดประสงค์ 0.25; คุณภาพ 0.50; วิธีตรวจและผลคาดหวัง 0.50; ปรับหรือคงข้อความพร้อมเหตุผล 0.50 ข้อ 1.2 รับ NFR ด้าน usability หากนิยามและวิธีวัดสอดคล้องกัน

ข้อ 2 ประเมินข้อเสนอโครงการ WorkshopHub (7 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 7-8; Lecture 2 หน้า 28-29

2.1 องค์ประกอบของข้อเสนอเดิม 4 คะแนน

องค์ประกอบ	คำตอบตัวอย่าง
Problem	มี: ใช้ฟอร์มแยกและรวบรวมเอง ทำให้สมัครซ้ำและยืนยันช้า
Users	มี: นักศึกษาและผู้ประสานงาน
Solution	มี: เว็บไซต์ให้ค้นหา สมัคร และจัดการรายชื่อ
Expected value	มีบางส่วน: คาดว่าจะลดข้อผิดพลาดและงานจัดการ แต่ยังไม่มีค่าตั้งต้นหรือเป้าวัดผล
Scope	มี: รุ่นแรกเฉพาะ workshop ฟรี; การชำระเงินและใบรับรองทำภายหลัง
Risk	ไม่มี: ยังไม่ระบุความเสี่ยงและวิธีรับมือ
Effort	ยังไม่มีปริมาณแรงงาน: “ก่อนภาคหน้า” เป็นกำหนดส่ง ไม่ใช่ประมาณการคน-วัน
Evidence of completion	ไม่มี: ยังไม่มีเกณฑ์หรือหลักฐานตรวจรับ

เกณฑ์ตรวจ: องค์ประกอบละ 0.50 ให้ตามสถานะและเหตุผลที่อิงข้อความ รวม 4 คะแนน

2.2 ข้อเสนอฉบับปรับปรุง 3 คะแนน

WorkshopHub จะรวมการสมัคร workshop ฟรีไว้ในเว็บไซต์เดียว แทนฟอร์มแยกที่ต้องรวบรวมเอง ให้นักศึกษาค้นหาและสมัครได้ ผู้ประสานงานจัดการรายชื่อได้ เพื่อลดการสมัครซ้ำและงานจัดการ รุ่นแรกไม่รวมการชำระเงินหรือใบรับรอง และตั้งเป้าเสร็จก่อนภาคหน้า **เสนอให้ยืนยันเพิ่ม:** วัดค่าตั้งต้นแล้วตกลงเป้าลดข้อผิดพลาดและเวลาทำงาน ตรวจการสมัครซ้ำบน server และทดสอบคำขอพร้อมกัน ให้ทีมแยกงานแล้วประเมินแรงงานเป็นคน-วันก่อนยืนยันกำหนดส่ง ส่วนการรับงานให้ใช้การสาธิต ผลทดสอบที่โยงเกณฑ์ยอมรับ ผลเทียบเป้าหมาย และผู้ประสานงานตรวจรับ

คำอธิบายเสริม: ข้อมูลยังไม่พอคำนวณ effort จริง จึงไม่กำหนดตัวเลขขึ้นเอง ข้อเสนอเรื่องตัวชี้วัด ความเสี่ยง วิธีประเมินแรงงานและหลักฐานรับงานยังต้องยืนยัน ไม่ใช่ข้อตกลงเดิม

เกณฑ์ตรวจ: คงปัญหา ผู้ใช้ วิธีแก้ คุณค่าและขอบเขตเดิม 1; เติมความเสี่ยงและแนวประเมิน effort โดยระบุสิ่งที่ยังต้องยืนยัน 1; หลักฐานรับงานตรวจได้ 1 รับประมาณการเชิงตัวเลขที่ระบุสมมติฐานและมีเหตุผลรองรับ แต่ไม่บังคับตัวเลขใด

ข้อ 3 สิทธิ์การใช้งานตามบทบาท (RBAC) (4 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 9; Lecture 5 หน้า 8, 19-23, 34

RBAC (Role-Based Access Control) คือกำหนดสิทธิ์ตามบทบาท; Student = นักศึกษา, Coordinator = ผู้ประสานงาน

การกระทำ	Student	Coordinator
ดู workshop ที่เผยแพร่	Allow	Allow
ส่งคำขอลงทะเบียนของตน	Allow	Deny
สร้าง workshop	Deny	Allow
แก้ไข workshop	Deny	Allow
ส่งออกรายชื่อ	Deny	Allow

3.2: ไม่พอ เพราะผู้ใช้เรียก API โดยไม่ผ่านปุ่มได้ server ต้องยืนยันตัวตน (authentication) แล้วตรวจสิทธิ์ (authorization) จากข้อมูลที่เชื่อถือได้ ใช้ default deny: ไม่ได้รับสิทธิ์ต้องปฏิเสธ ไม่เชื่อ role ที่แนบมากับคำขอ

คำอธิบายเสริม: ตัวอย่างทดสอบที่เสนอ: ใช้บัญชี Student เรียกส่งออกรายชื่อโดยตรง แม้ปลอม role=Coordinator ต้องได้ 403 และไม่ได้รับรายชื่อ; รหัสตอบกลับนี้เป็นแบบที่เสนอ ไม่ใช่ API contract ที่โจทย์ให้

เกณฑ์ตรวจ: 3.1 ช่องละ 0.25 รวม 2.5; 3.2 แยก UI/การบังคับสิทธิ์ 0.5, ตรวจสิทธิ์ที่เชื่อถือได้และ default deny 0.5, ทดสอบตรงพร้อมผลคาดหวัง 0.5 ไม่เพิ่ม inheritance หรือกฎเจ้าของที่โจทย์ไม่ได้ให้

ข้อ 4 วิเคราะห์การอัปโหลดไฟล์ (6 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 10-11; Lecture 5 หน้า 5-8, 28-38

4.1 คำตอบตัวอย่าง (3 คะแนน)

ประเด็น	คำตอบ	คะแนน
Attacker	ผู้โจมตีคือนักศึกษาที่ข้ามการตรวจใน browser	0.5
Assets	สิ่งที่ต้องปกป้องคือรายงานผู้อื่นและ server	0.5
Attack surface	จุดโจมตีคือ API อัปโหลดและ URL ของไฟล์	0.5
Trust boundary	ข้ามขอบเขตความเชื่อถือจาก client สู่ server และจากไฟล์ข้อมูลเป็นโค้ดที่รันได้	1
Security objective	Confidentiality (การรักษาความลับ) ถูกละเมิด เพราะอ่านรายงานผู้อื่นได้	0.5

4.2 มาตรการควบคุม (control) และการทดสอบเชิงลบ (negative test) คู่ละ 1.5 คะแนน

Control	วิธีทดสอบ	ผลที่คาดหวัง
ตรวจชนิดและโครงสร้าง PDF บน server ไม่เชื่อชื่อไฟล์หรือ MIME อย่างเดียว	ส่งสคริปต์ปลอม PDF เข้า API โดยข้าม browser	ปฏิเสธไฟล์และไม่มี URL ที่ใช้รันสคริปต์นั้นได้
เก็บในพื้นที่ที่รันโค้ดไม่ได้ และตรวจสิทธิ์ก่อนดาวน์โหลด	เปิด URL ของไฟล์ทดสอบ แล้วใช้ A ขอรายงานของ B	สคริปต์ไม่ทำงาน และ A ไม่ได้ข้อมูลของ B หากไม่มีสิทธิ์

เกณฑ์ตรวจ 4.2: คู่ละมาตรการ 0.5 + วิธีทดสอบ 0.5 + ผลคาดหวัง 0.5 ตรวจเฉพาะ browser ยังไม่พอ

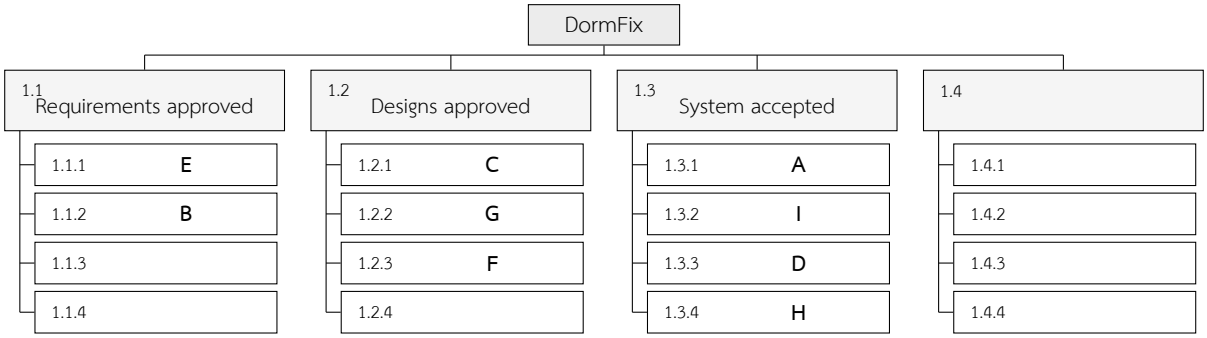
คำอธิบายเสริม: เหตุการณ์นี้เป็นโค้ดรันบน server ไม่ใช่ XSS ที่รันใน browser; integrity (ความถูกต้องของข้อมูล) และ availability (ความพร้อมใช้งาน) อาจได้รับผล แต่โจทย์ยังไม่ยืนยันว่าเสียหายแล้ว

ข้อ 5 วางแผน DormFix **(10 คะแนน)**

แหล่งตรวจ: Lecture 10 หน้า 14-21; Lecture 6 หน้า 71-96; RACI ยึดบันทึกประชุมข้อ 5.2 ของชุดฝึก

WBS (Work Breakdown Structure) ใช้แบ่งงานเป็นลำดับชั้น; milestone คือจุดสำเร็จ ไม่ใช่งานที่ใช้เวลาทำ

5.1 WBS ตามผลสำเร็จ **3 คะแนน**



เกณฑ์ตรวจ: milestone/ลำดับ 1; A-I ครบและอยู่กลุ่มถูก 1; เรียงงานภายในกลุ่มถูกและเว้นช่องไม่ใช้ 1 งาน C/G และ A/I ทำขนานกัน จึงสลับลำดับในคู่ได้ เส้น WBS แสดงลำดับชั้น ไม่ใช่ dependency

5.2 RACI **2 คะแนน**

R = Responsible (ลงมือทำ), A = Accountable (รับผิดชอบผล), C = Consulted (ให้คำปรึกษา), I = Informed (รับแจ้ง); ไม่เกี่ยวข้องใส่ “-”

งาน	PM	Aom	Bank	Chai	Dao
E	A	R	C	I	-
C	A	C	R	I	-
G	A	-	C	R	I
D	A	I	-	C	R

เกณฑ์ตรวจ: 20 ช่อง ช่องละ 0.10 ตามบันทึกประชุม รวม 2 คะแนน

5.3 กำหนดการและคำอธิบาย **5 คะแนน**

E → B → {C,G} → F → {A,I} → D → H → project buffer

เหตุผลสั้น: เผื่อ task buffer หลัง B/F/H อย่างละ 1 วัน เพราะรอคนนอกทีมที่มีงานอื่น ส่วนงานภายในใช้ project buffer 2 วันท้ายร่วมกัน; milestone ใช้เวลา 0 วันเพราะเป็นจุดบรรลุผล

งาน	ทำงาน (ม.ค. 2027)	Buffer	เริ่ม-สิ้นสุดรวม buffer
E	4, 5, 6	ไม่มี	4-6
B	7	8	7-8
C	11, 12, 13	ไม่มี	11-13
G	11, 12	ไม่มี	11-12
F	14	15	14-15
A	18, 19, 20, 21	ไม่มี	18-21
I	18, 19, 20	ไม่มี	18-20
D	22, 25	ไม่มี	22-25
H	26	27	26-27
PB	-	28, 29	28-29

ข้อ 5.3: Gantt chart ตัวอย่าง มกราคม 2027

Milestones: M1 = 8, M2 = 15, M3 = 29 ม.ค. 2027; จบวันที่ 29 รวม 20 วันทำงานพร้อม buffer Chai ทำ G ก่อน A และ Dao ทำ I ก่อน D จึงไม่ใช้คนซ้อนกัน; D เว้นเสาร์-อาทิตย์ที่ 23-24 ม.ค.

เกณฑ์ตรวจ 5.3: No./Owner 0.5; ระยะเวลา/วัน/Start-Due 1; dependency/resource 1; buffers 1; milestone 0.5; เหตุผล buffer 0.5 และ milestone ไม่เพิ่มเวลา 0.5 รวม 5 ไม่บวกคะแนนคำอธิบายซ้ำ

สัปดาห์ที่ 1-2

No.	Task	Owner	Start	Due	4	5	6	7	8	11	12	13	14	15
1.1	M1	-	8	8					*					
1.1.1	E	Aom	4	6	X	X	X							
1.1.2	B	PM	7	8				X	-					
1.2	M2	-	15	15										*
1.2.1	C	Bank	11	13						X	X	X		
1.2.2	G	Chai	11	12						X	X			
1.2.3	F	PM	14	15									X	-
														ⓐ

สัปดาห์ที่ 3-4

No.	Task	Owner	Start	Due	18	19	20	21	22	25	26	27	28	29
1.3	M3	-	29	29										*
1.3.1	A	Chai	18	21	X	X	X	X						
1.3.2	I	Dao	18	20	X	X	X							
1.3.3	D	Dao	22	25					X	X				
1.3.4	H	PM	26	27							X	-		
-	PB	-	28	29									-	-
					ⓐ									

X = วันทำงาน - = buffer * = milestone PB = project buffer

จุดเชื่อม a แสดงว่า A และ I ต้องรอ F กับ buffer ของ F ก่อนเริ่ม

M1: ข้อกำหนดอนุมัติ M2: แบบอนุมัติ M3: ตรวจรับระบบรวม project buffer

คำอธิบายเสริม: buffer และวันครบกำหนดนี้ใช้ตามเงื่อนไข DormFix ไม่ใช่กฎว่าทุกโครงการต้องเผื่อหลังงานอนุมัติทุกครั้ง

ข้อ 6 ปรับรายละเอียด Backlog (10 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 22-23; Lecture 7 หน้า 60-64, 94-104

6.1 User story: ใครต้องการอะไร เพื่ออะไร 2 คะแนน

“ในฐานะนักศึกษา ฉันต้องการจองสนามแบดมินตันครั้งละ 1 ชั่วโมง เพื่อไม่ต้องต่อคิวที่เคาน์เตอร์”

เกณฑ์ตรวจ: ผู้ใช้ 0.5; ความสามารถ 0.75; คุณค่า 0.75

6.2 รายละเอียดจากลูกค้า 2 คะแนน

มีข้อมูลสนามและเวลาเปิดแล้ว; จองครั้งละ 1 ชั่วโมง; แต่ละคนถือการจองได้ไม่เกิน 1 รายการต่อวัน; จองสำเร็จแล้วแสดง Booked ส่วนการยกเลิกและการนับสิทธิ์ วันจองล่วงหน้าและการชำระเงินยังต้องถาม

เกณฑ์ตรวจ: ข้อเท็จจริงสี่ประเด็น ประเด็นละ 0.5

6.3 เกณฑ์ยอมรับสองสถานการณ์ 4 คะแนน

สถานการณ์: จองสำเร็จ
Given นักศึกษา S ยังไม่จองวันนี้ และสนาม 2 ว่างเวลา 17:00-18:00 ซึ่งเป็นช่วงให้บริการ
When S จองสนาม 2 ช่วงนั้น
Then ระบบยืนยันการจอง 1 ชั่วโมงให้ S และแสดงช่วงนั้นเป็น Booked

สถานการณ์: เกินสิทธิ์ต่อวัน
Given S มีการจองวันนี้แล้ว 1 รายการ และมีอีกช่วงที่ว่าง
When S ขอจองเพิ่มวันนี้
Then ปฏิเสธคำขอ จำนวนการจองไม่เพิ่ม และช่วงใหม่ยังว่าง

เกณฑ์ตรวจต่อสถานการณ์: เงื่อนไข 0.5; การกระทำ 0.5; ผลและสถานะตามกฎ 1 รวมสองสถานการณ์ 4

6.4 ข้อมูลกำกับ Issue (Metadata) 2 คะแนน

Assignees: -; Labels: enhancement; Milestone: -; Estimate: -

เกณฑ์ตรวจ: ช่องละ 0.5 ไม่เดาเจ้าของ/sprint/points และไม่ใช้ 0 แทนค่ายังไม่ทราบ

ข้อ 7 วางแผน Sprint (5 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 25-26; Lecture 7 หน้า 67-72, 114-118

งาน	เรื่อง	Points	เลือก	เหตุผล
1	ดูตำแหน่งรถปัจจุบัน	8	Yes	ตรงเป้าหมายและสำคัญสูง
2	จ่ายด้วย QR	3	No	ไม่ตรงเป้าหมายรอบนี้
3	เวลาถึงป้ายของฉัน	5	Yes	ตรงเป้าหมาย
4	เตือนก่อนรถถึง	3	Yes	ช่วยเป้าหมายและใส่ได้
5	เวลาถึงทุกป้าย	5	No	ลำดับต่ำกว่าและเกินกำลังทีม
6	แก้รายการถูกตัดบนมือถือ	2	No	ไม่เหลือกำลังรับงานเพิ่ม

7.2: $8 + 5 + 3 = 16$ points **7.3:** เลือก 1/3/4 ตามเป้าหมาย Sprint Goal และลำดับความสำคัญ รวม 16 points พอดีกำลังทีม (capacity); ไม่เลือก 2 เพราะไม่ช่วยเป้าหมาย ส่วน 5/6 เกินกำลังที่เหลือ

เกณฑ์ตรวจ: points หกช่อง ช่องละ 0.25 และการเลือกหกช่อง ช่องละ 0.25 รวม 3; ผลรวม 1; เหตุผลด้านเป้าหมาย/ลำดับ 0.5 และ capacity 0.5

คำอธิบายเสริม: เทียบ anchor 3 บน scale: ใหญ่ขึ้นสองขั้นคือ 3 ไป 5 ไป 8 ไม่ใช่ 3+2; points วัดขนาดงานสัมพัทธ์ ไม่ใช่วันหรือชั่วโมง รับแผนอื่นที่มีเหตุผลจากโจทย์และไม่เกิน capacity

ข้อ 8 ความสัมพันธ์ใน Use Case Diagram (10 คะแนน)

แหล่งตรวจ: ข้อกำหนด R1-R8 และ R12 ในชุดฝึกนี้; Lecture 4 หน้า 5-8 เรื่อง actor/association และ include/extend/generalization

คำตอบย่อ: เชื่อม actor กับความสามารถที่ใช้โดยตรง ใช้ include กับงานย่อยที่ต้องทำเสมอ ใช้ extend กับการแนบไฟล์/ย้าย owner ที่เกิดเฉพาะเงื่อนไข และให้ Administrator เป็นชนิดย่อยของ IT Staff ตาม R1 ไม่ลากลูกศรให้เป็นลำดับการทำงาน

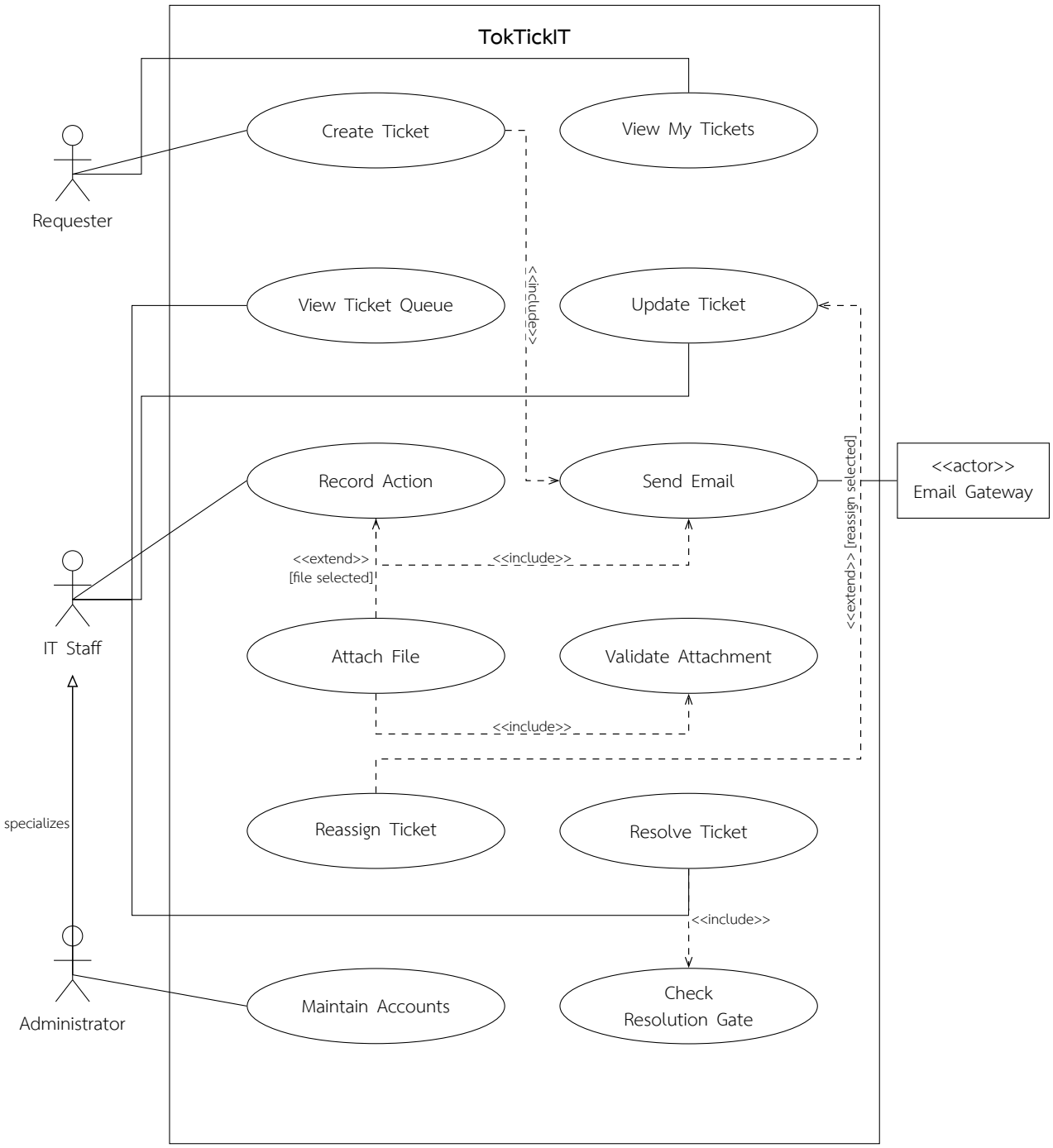
ชนิด	คู่ความสัมพันธ์ที่ต้องมี
Association (เส้นทึบ)	Requester -- Create Ticket; Requester -- View My Tickets; IT Staff -- View Ticket Queue, Update Ticket, Record Action และ Resolve Ticket; Administrator -- Maintain Accounts; Email Gateway -- Send Email
Include (เส้นประ)	Create Ticket → Send Email; Record Action → Send Email; Attach File → Validate Attachment; Resolve Ticket → Check Resolution Gate
Extend (เส้นประ)	Attach File → Record Action [เลือกไฟล์]; Reassign Ticket → Update Ticket [เลือกย้าย owner]
Generalization	Administrator → IT Staff ใช้เส้นทึบหัวสามเหลี่ยมโปร่งชี้ไปชนิดทั่วไป

เหตุผล: Include ชี้จาก use case หลักไปงานที่ต้องเรียก ส่วน extend ชี้จากพฤติกรรมเสริมกลับไปยังฐานที่ยังทำงานได้โดยไม่มีส่วนเสริม ไม่สลับทิศเพราะอ่านตามลำดับข้อความในโจทย์

เกณฑ์ตรวจ: associations/ขอบเขตครบ 3; include สี่คู่ คู่ละ 0.75 รวม 3; extend สองคู่พร้อมเงื่อนไข คู่ละ 1 รวม 2; generalization ถูกคู่และทิศทาง 2 รวม 10 ให้บางส่วนเมื่อสื่อความหมายถูกแต่ขาดป้ายหรือทิศทาง

ขอบเขต: ไม่เพิ่ม Login เพราะโจทย์กำหนดเป็น precondition และไม่เพิ่ม Public Comment/Internal Note เป็น nodes ใหม่ เนื่องจาก R12 ให้เติมเฉพาะโครง ความสัมพันธ์ของ Administrator ในข้อนี้เป็นกฎของโจทย์ฝึก ไม่ใช่ข้อสรุปว่า RBAC ทุกระบบต้องมี role inheritance

ข้อ 8: แผนภาพตัวอย่าง



อ่านภาพ: เส้น association ไม่มีหัวลูกศร ไม่ตีความว่า actor เป็นข้อมูลที่ไหลผ่านระบบ เส้นที่ตัดกันไม่ใช่ junction; ปรับเส้นทางวาดได้หากคู่และชนิดความสัมพันธ์ยังตรงตาราง

ข้อ 9 State Diagram ของ Ticket **(10 คะแนน)**

แหล่งตรวจ: R8-R10 ในชุดฝึกนี้; Lecture 4 หน้า 45-46 เรื่อง event/guard/state transition

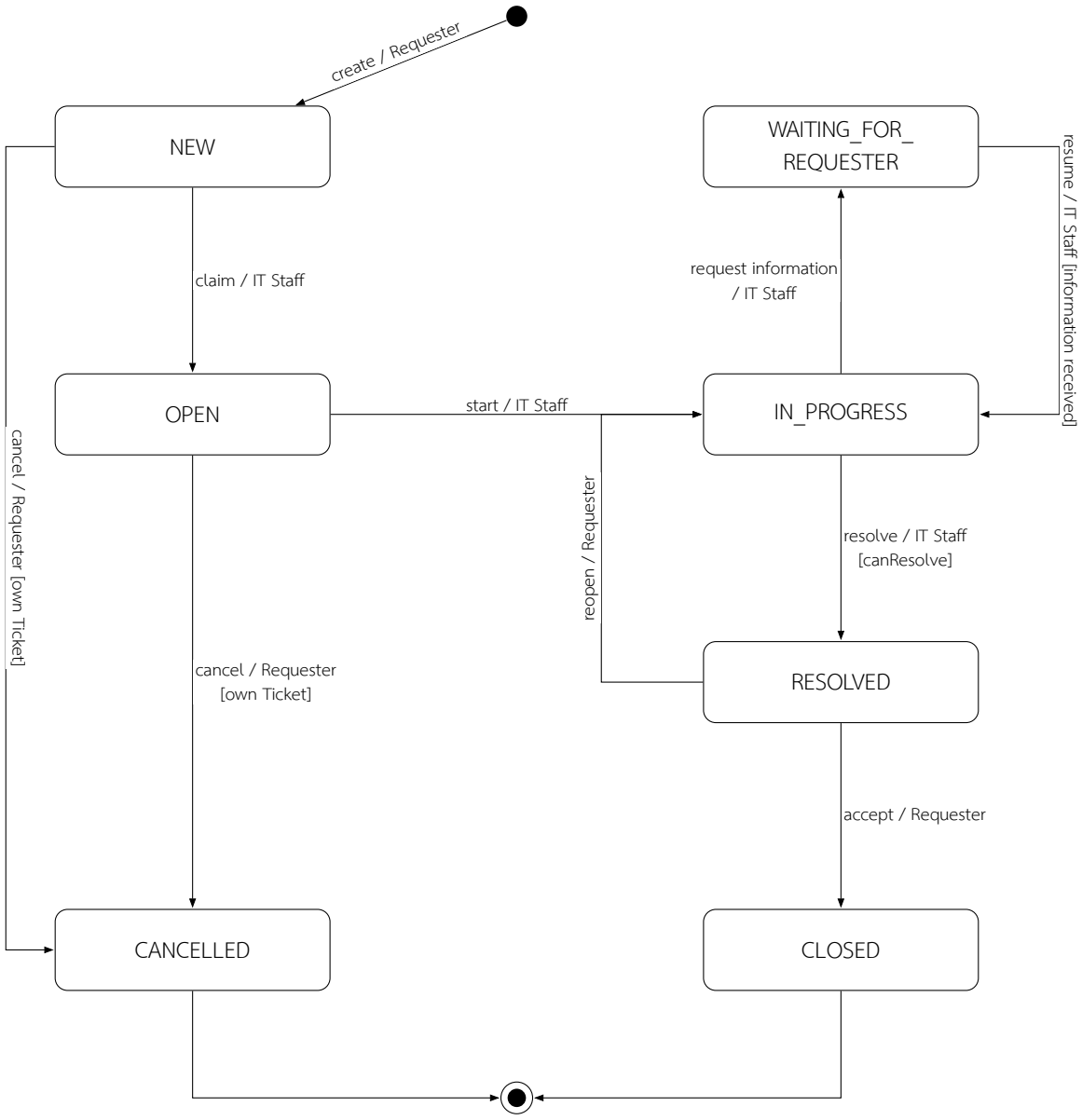
คำตอบย่อ: แสดงเฉพาะเส้นทางที่ข้อกำหนดอนุญาต แยกเหตุการณ์กับเงื่อนไขบนลูกศร และไม่สร้างสถานะ Error เมื่อการกระทำถูกปฏิเสธ

จาก → ไป	Event / ผู้กระทำ	Guard หรือผลสำคัญ
Initial → NEW	create / Requester	สร้าง Ticket สำเร็จ
NEW → OPEN	claim / IT Staff	กำหนด owner
OPEN → IN_PROGRESS	start / IT Staff	เริ่มงาน
IN_PROGRESS → WAITING_FOR_REQUESTER	request information / IT Staff	รอข้อมูล
WAITING_FOR_REQUESTER → IN_PROGRESS	resume / IT Staff	ได้รับข้อมูลแล้ว
IN_PROGRESS → RESOLVED	resolve / IT Staff	[canResolve] ตาม R8
RESOLVED → CLOSED	accept / Requester ของ Ticket	ยอมรับผล
RESOLVED → IN_PROGRESS	reopen / Requester ของ Ticket	ต้องแก้ต่อ
NEW หรือ OPEN → CANCELLED	cancel / Requester ของ Ticket	เป็น Ticket ของตน
CLOSED และ CANCELLED → Final	จบวงจร	ไม่มี transition กลับมาทำงาน

canResolve: owner ไม่ว่าง **และ** มี COMPLETED Action อย่างน้อยหนึ่งรายการ **และ** จำนวน OPEN Actions เป็นศูนย์ ถ้าไม่ผ่านข้อใดให้คงสถานะเดิมและแจ้งเหตุผล การเขียน Comment ไม่ใช่เหตุให้เปลี่ยน status โดยอัตโนมัติ

เกณฑ์ตรวจ: Initial/NEW และ terminal exits 1; claim/start 2; request information/resume 2; resolve พร้อม guard ทั้งสาม 2; accept/reopen 2; cancel เฉพาะ NEW/OPEN ของตน 1 รวม 10 ไม่ให้คะแนนเส้นที่เพิ่ม transition ต้องห้าม

ข้อ 9: แผนภาพตัวอย่าง



canResolve = owner exists AND completedActions ≥ 1 AND openActions $= 0$.

จุดที่ห้ามเติมเอง: ไม่ยกเลิกจาก IN_PROGRESS/RESOLVED ไม่เปิด CLOSED/CANCELLED กลับมา และไม่ใช้เส้น include/extend ใน State Diagram

ข้อ 10 Ticket Queue และ Staff Ticket Detail (10 คะแนน)

แหล่งตรวจ: R2-R4 และ R7-R11 ในชุดฝึกนี้; Lecture 4 หน้า 111-117 เรื่อง List/Form UI notation

10.1 List View 4 คะแนน

คำตอบย่อ: มีช่องค้นหา title และกรอง status, IT priority, owner; แสดงครบทั้งแปด fields ใน R4 เลือกเปิดรายการและเปลี่ยนหน้าได้ แยก empty กับ no-results โดยคงตัวกรองให้แก้ได้

TokTickIT / Ticket Queue

Search title: WiFi [Search] [Clear filters]

Status: [All v] IT priority: [All v] Owner: [All staff v]

Ticket No.	Title	Requester	Status	Requested priority	IT priority	Owner	Updated
TT0042	WiFi in Lab	Pim	IN_ PROGRESS	HIGH	MEDIUM	Pat	27 Oct 13:00

[Open selected Ticket] [Previous] Page 1 / 4 [Next]

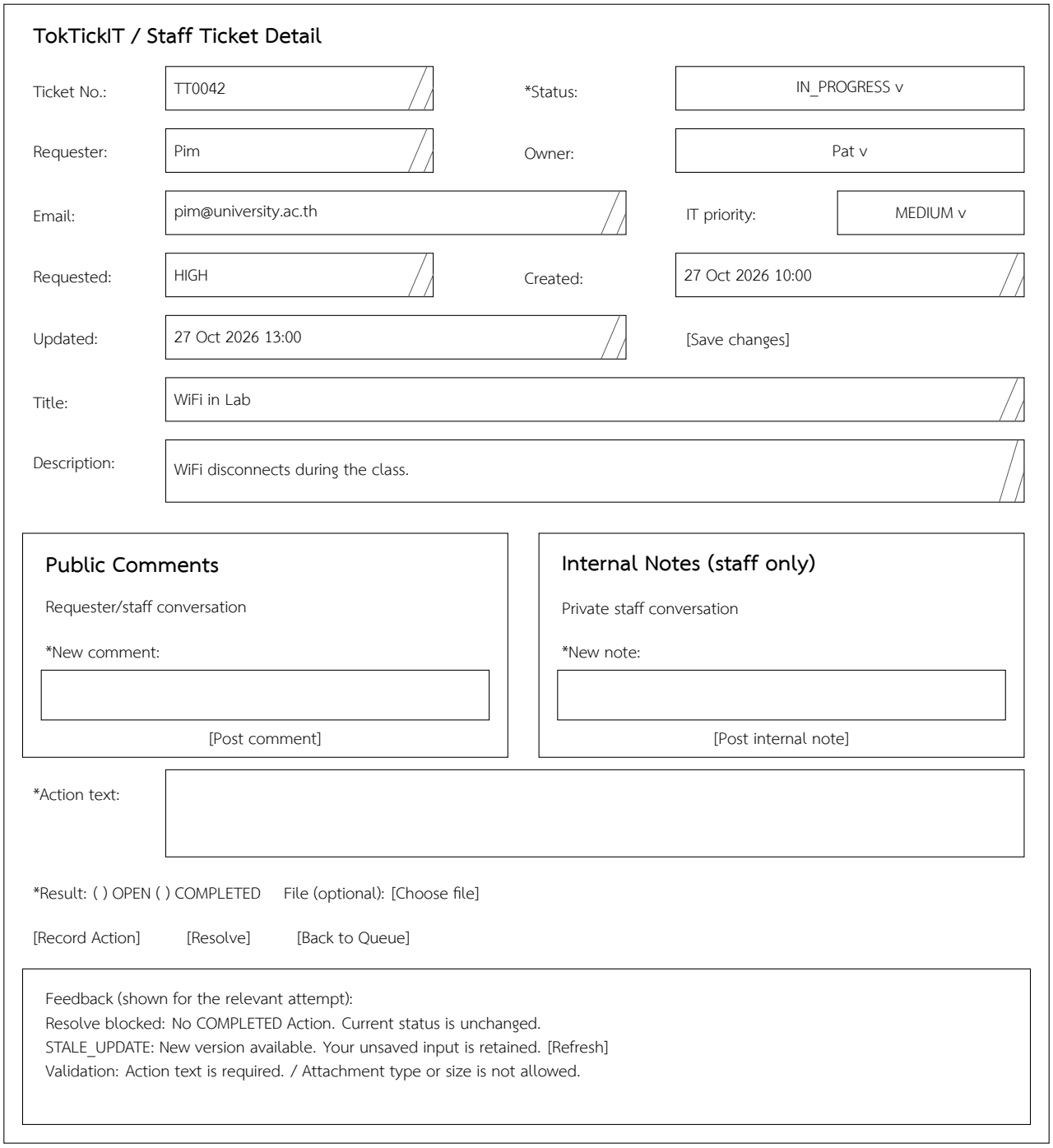
State messages (display only the applicable one):
Empty: No Tickets have been created.
No results: No Tickets match these filters. [Clear filters]

เกณฑ์ตรวจ: fields ครบแปดช่อง 2; ค้นหาและตัวกรองครบ 1; เลือกเปิด/pagination 0.5; empty กับ no-results แยกกัน 0.5 รวม 4 ยอมรับ card/detail หากข้อมูลครบ ไม่จำเป็นต้องวางทุก field ในแถวแคบเดียว

Notation: ผลลัพธ์ในตารางอ่านอย่างเดียว ช่องกรองแก้ได้; ใน Form ใช้เส้นเฉียงสำหรับค่าจากระบบและเก็บ * ไว้เฉพาะ input ที่ต้องกรอกจริง ไม่เติม * ให้ปุ่มหรือทุกคอลัมน์

10.2 Form View **6 คะแนน**

คำตอบย่อ: แยกข้อมูลเดิม/ข้อมูลระบบเป็น read-only จาก controls ของ IT Staff แยก public/internal ชัดเจน และเก็บข้อความที่กำลังกรอกเมื่อ stale



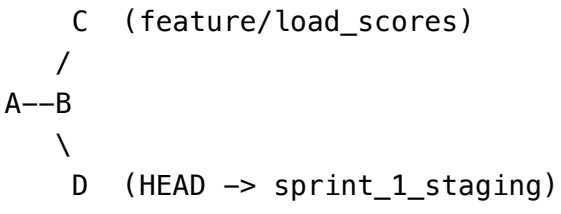
เกณฑ์ตรวจ: data/read-only และ controls ตาม R3 ครบ 2; แยก Public Comments/Internal Notes 1; Action/optional file พร้อม validation 1; Resolve gate/refusal 1; stale แล้วคง input/Refresh ไม่ส่งเอง 1 รวม 6

เงื่อนไข: แสดง panel ข้อผิดพลาดตามเหตุการณ์จริง ไม่บังคับให้หลาย error เกิดพร้อมกัน; Requester ไม่เห็น Internal Notes หรือจำนวนรายการ และ role/state ที่ไม่มีสิทธิ์ต้องไม่เกิดการเปลี่ยนแปลง

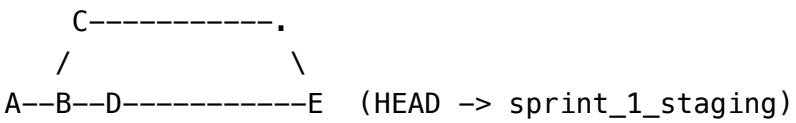
ข้อ 11 Switch ทำ commit แยกสาย แล้ว merge (10 คะแนน)

แหล่งตรวจ: โจทย์สถานการณ์ดัดแปลงในชุดฝึก; หลัก three-way merge และ refs: Samples หน้า 6, 12, 21

ก่อน merge: C และ D ต่างมี parent เป็น B เพราะทำงานต่อจาก B คนละ branch ไม่ใช่ C ตามด้วย D ในเส้นเดียว HEAD อยู่ sprint_1_staging ที่ D หลัง switch ครั้งที่สอง



หลัง merge: E เป็น merge commit ที่มี first parent D และ second parent C; เลื่อนเฉพาะ sprint_1_staging ไป E โดย HEAD ยังชี้ branch เดิม



Reference	ก่อน merge	หลัง merge
main	B	B
feature/load_scores	C	C
sprint_1_staging	D	E
origin/main; origin/sprint_1_staging; origin/feature/load_scores	B ทุกตัว	B ทุกตัว
HEAD	sprint_1_staging (D)	sprint_1_staging (E)

เหตุผล: เป็น three-way merge เพราะ C กับ D แยกจาก common ancestor B ไม่มีปลายใดเป็น ancestor ของอีกปลาย จึง fast-forward ไม่ได้ source branch คงที่ C และ origin/* คงที่ B เพราะไม่มี fetch หรือ push การ switch เปลี่ยน current branch/working tree แต่ไม่สร้าง commit; C และ D เกิดจาก commit สองคำสั่ง ส่วน E เกิดจาก merge

เกณฑ์ตรวจ 10 คะแนน: C และ D มี parent B ครบ 2; local branches/HEAD ก่อน merge ถูก 2; E มี parents D และ C ครบ 2; current branch/HEAD หลัง merge ถูก 1; คง main, source branch และ remote refs ถูก 2; อธิบาย three-way และ switch ไม่สร้าง commit 1

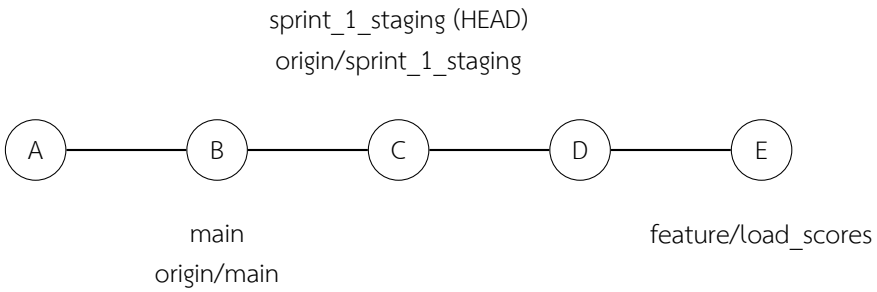
รูปแบบคำตอบ: รับภาพวงกลมกับเส้นเชื่อมหรือ ASCII ที่สื่อ ancestry เดียวกัน อย่าเติม commit ใหม่เพียงเพราะมี switch และอย่าเลื่อน origin/* ตาม local branch โดยไม่มีการติดต่อ remote

ข้อ 12 คำสั่งและ graph แบบต่อเนื่อง **(10 คะแนน)**

แหล่งตรวจ: Samples ข้อ 2.3 หน้า 10-11 และเฉลยหน้า 20-21; สมมติฐาน Git หน้า 6

12.1 Switch **2 คะแนน**

```
git switch sprint_1_staging
```

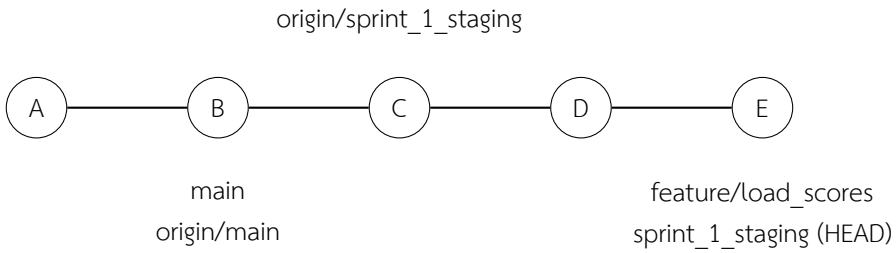


คะแนน: คำสั่ง 1; graph พร้อม HEAD ที่ C และ pointers เดิม 1

12.2 Merge **3 คะแนน**

```
git merge feature/load_scores
```

เป็น fast-forward เพราะ C เป็น ancestor ของ E ไม่สร้าง merge commit ใหม่



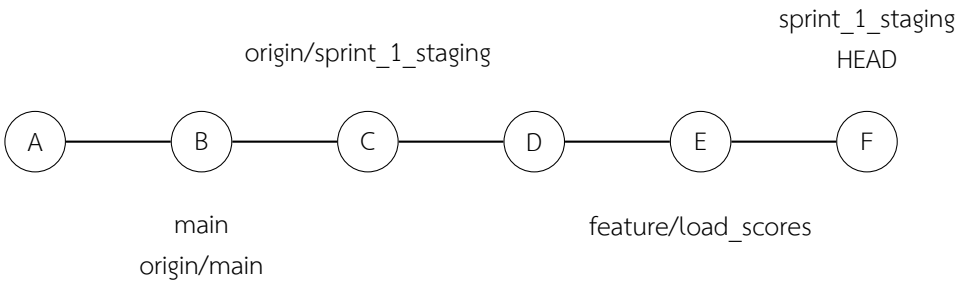
คะแนน: คำสั่ง merge ถูก branch 1; ระบุประเภท fast-forward 1; graph เลื่อน local sprint/HEAD ไป E และคง origin/sprint ที่ C 1

ข้อควรระวัง: กราฟเริ่มต้นไม่มี origin/feature/load_scores จึงไม่เติมตัวชี้นี้เอง และไม่ใช้ `git merge --no-ff` เพราะจะสร้าง merge commit ต่างจาก fast-forward ตามโจทย์

12.3 Inspect, stage และ commit **5 คะแนน**

```
git status
git diff -- score_data.py scores.csv
git add -- score_data.py scores.csv validation_notes.txt
git diff --cached -- score_data.py scores.csv validation_notes.txt
git commit -m "Document score validation cases"
```

เหตุผลสั้น: `status` ดูสถานะไฟล์; `diff` ดูส่วนต่างของไฟล์ที่ Git ติดตามแล้ว; `diff --cached` ตรวจสิ่งที่เลือกไว้หลัง stage รวมไฟล์ใหม่ ก่อนสร้าง commit รับวิธีอื่นที่ตรวจครบและเลือกเฉพาะสามไฟล์



F มี parent E ตัวเดียวและบันทึกการแก้ทั้งสามไฟล์; sprint_1_staging = F โดย HEAD ชี้ branch นี้ ส่วน feature/load_scores = E, main และ origin/main = B, origin/sprint_1_staging = C เพราะไม่มีคำสั่งติดต่อ remote จึงคง remote-tracking refs เดิม

เกณฑ์ตรวจ 5 คะแนน:

- Inspect เห็นสถานะและเนื้อหาการเปลี่ยนแปลงที่จะ commit รวมไฟล์ใหม่ 1
- Stage ชื่อไฟล์สามรายการชัดเจน ไม่ใช้ `git add .` 1
- Commit ด้วยข้อความที่กำหนด 1
- F ต่อจาก E และ local sprint/HEAD ไป F 1
- pointers อื่นคงครบและไม่สร้าง merge commit แทรก 1

หากกราฟผิดสืบเนื่องจาก 12.2 ให้ตรวจขั้นใหม่บน branch และ ancestry ที่ผู้ตอบวาดอย่างสอดคล้อง ให้คะแนนทักษะใหม่ที่ถูกต้อง ไม่หักต้นเหตุเดิมซ้ำทั้งข้อ

แหล่งอ้างอิงของเฉลย

ชื่อ Lecture/Samples ที่อ้างรายข้อหมายถึงไฟล์ด้านล่าง เลขหน้าคือลำดับหน้า PDF ไม่ใช่เลขสไลด์

Lecture 2: CPE334_Lecture02 Requirement Engineering.pdf

Lecture 4: CPE334_Lecture04 UML, SDS Features, STS Features.pdf

Lecture 5: CPE334_Lecture05 - Security in SE.pdf

Lecture 6: CPE334_Lecture06 - Project Management.pdf

Lecture 7: CPE334_Lecture07 - Project Execution.pdf

Lecture 10: CPE334_Lecture10 - Midterm Exam Prep_Group Work Project.pdf

Samples: CPE334_Midterm Samples by Suthep.pdf

เฉลยนี้ใช้ข้อความและหน้าที่อ้าง ไม่ได้ถอดทุกภาพเฉลยหรือเอกสารในลิงก์ภายนอก แผนภาพตัวอย่างวาดขึ้นสำหรับชุดฝึก ไม่ใช่ภาพเฉลยทางการ

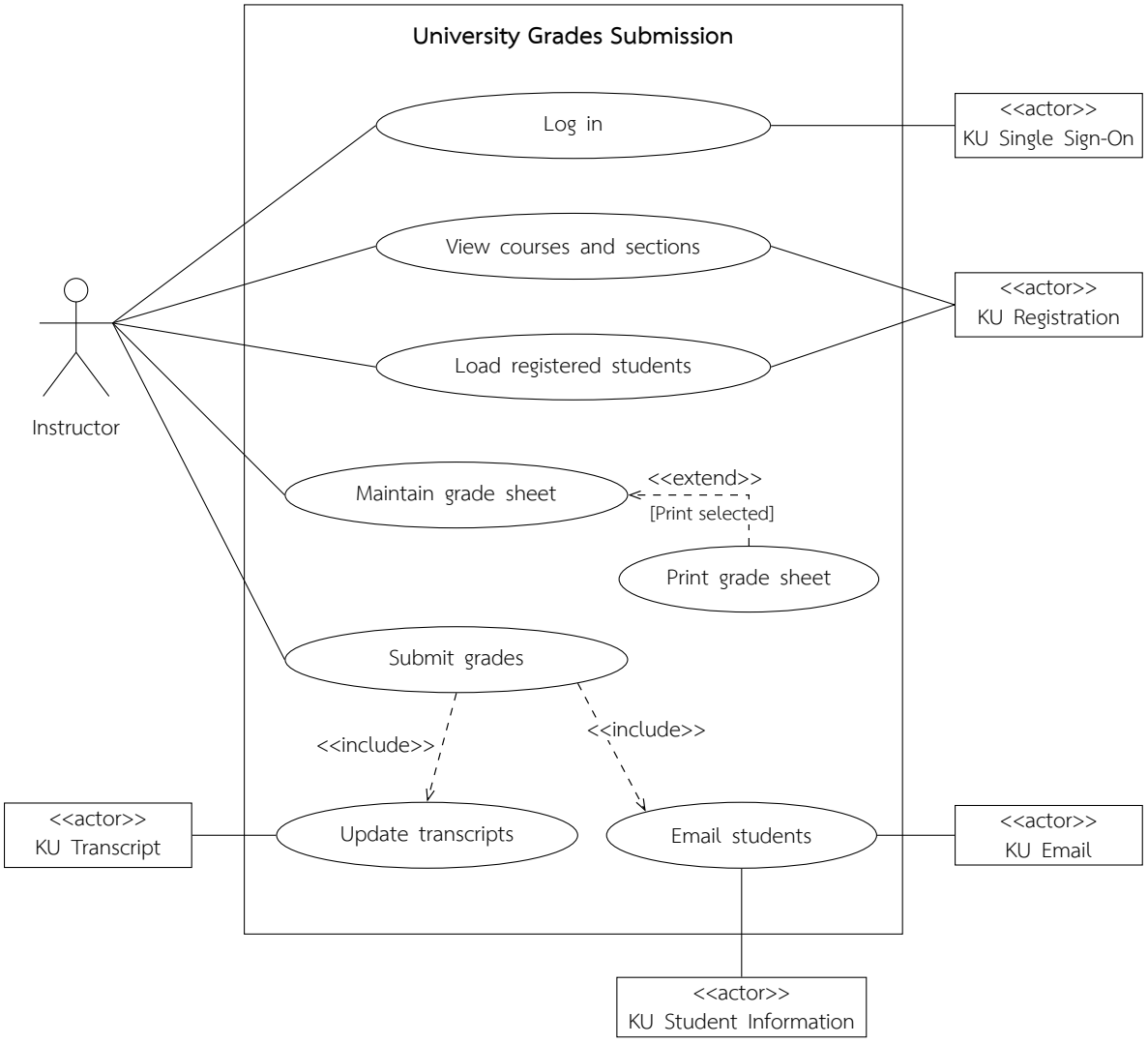
เฉลยแบบฝึกเสริม S1-S3

แบบฝึกเหล่านี้ไม่รวมใน 100 คะแนนหรือเวลา 3 ชั่วโมง ใช้รายการตรวจตนเองแทนการเพิ่มคะแนนให้ชุดหลัก

S1 Use Case: University Grades Submission

แหล่งตรวจ: Lecture 4 หน้า 5-8 (notation) และ 64-65 (requirements); ภาพประยุกต์สำหรับแบบฝึกนี้

คำตอบย่อ: Instructor เป็นผู้ใช้หลัก ระบบภายนอกทั้งห้าอยู่นอก boundary การ submit ต้องอยู่ใน Ready และต้องอัปเดต transcript/แจ้ง email ทั้งคู่ ส่วน Print เป็นทางเลือกของการดูแล grade sheet



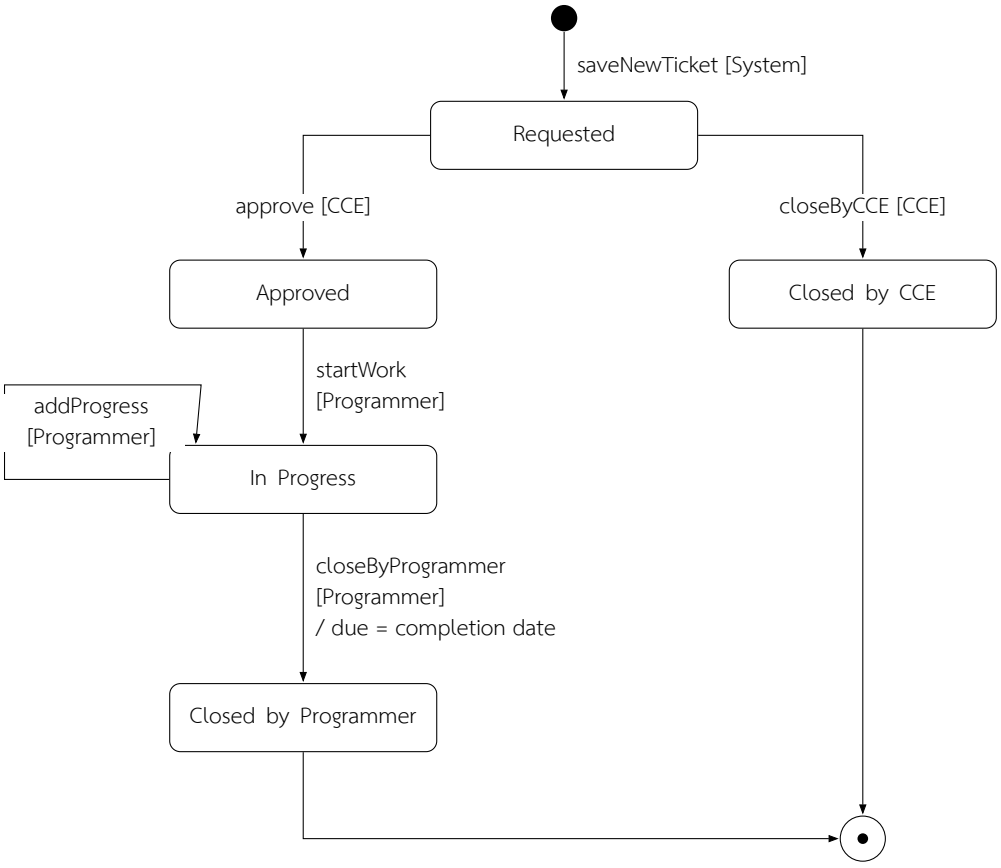
ตรวจตนเอง: actor association เป็นเส้นทึบไม่มีหัว; **<<include>>** ชี้จาก Submit ไปงานบังคับ (Update transcript/Email students); **<<extend>>** ชี้จาก Print ไป Maintain พร้อมเงื่อนไข [Print selected]. Generalization ต้องชี้จาก child ไป parent ด้วยเส้นทึบและสามเหลี่ยมโปร่งที่ parent แต่โจทย์ Grades ไม่มี actor/use case แบบ parent-child จึงไม่วาดเพิ่ม

อ่านภาพให้ถูก: ไม่ใช่ลำดับขั้นตอนการทำงาน Login เป็นเงื่อนไขก่อนใช้ฟังก์ชัน ไม่จำเป็นต้องลาก include ไป Login จากทุก use case; Student Info ให้ข้อมูลติดต่อแก่การแจ้ง email ส่วน KU Email เป็นผู้ส่ง

S2 State: Service Ticket

แบบฝึกเสริม ไม่รวม 100 คะแนน

แหล่งตรวจ: Lecture 4 หน้า 45-46 และเส้นทางภาพตัวอย่างหน้า 61-62



คำตอบย่อ: เริ่ม Requested แล้ว CCE เลือกอนุมัติหรือปิด หากอนุมัติ Programmer เริ่มเป็น In Progress เพิ่ม progress แบบ self-loop และปิดเมื่อเสร็จ ทั้งสองสถานะ Closed จบวงจรและไม่มี actions ใหม่

จุดตรวจ	สิ่งที่ต้องมี
เริ่มวงจร	save สร้าง Action แรกด้วย Requested และ Initial Due Date
บทบาท	CCE ทำเฉพาะ Requested; Programmer ใช้เส้นทาง Approved/In Progress
Self-loop	progress notes/วันนัดใหม่ไม่ทำให้หลุดจาก In Progress
ผลของ action	อัปเดต Last Status/Last Due Date; action เดิมแก้หรือลบไม่ได้
สิ้นสุด	Closed ทั้งสองแบบไป final และไม่มีเส้นทางกลับไปเพิ่ม action

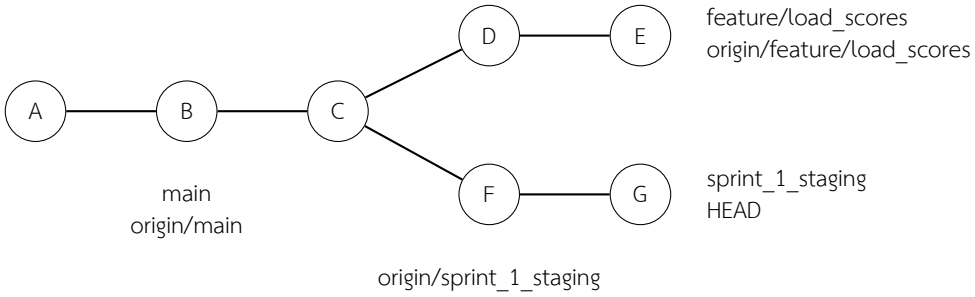
ขอบเขตภาพ: โจทย์นี้ยึดเส้นทางเริ่มงานก่อนปิดจากตัวอย่างหน้า 62 ไม่เพิ่มทางลัด Approved ไป Closed by Programmer จากการตีความข้อความหน้า 46 ที่กว้างกว่า

Notation: วงเล็บเหลี่ยมในข้อนี้ระบุ actor ตามคำสั่งหน้า 61 ไม่ใช่ Boolean guard ของ UML ทั่วไป ไม่ควรนำรูปแบบนี้ไปใช้แทน guard โดยไม่อธิบายบริบท

S3 Git: Explain the repository graph

แบบฝึกเสริม ไม่รวม 100 คะแนน

แหล่งตรวจ: Samples ข้อ 2.2 หน้า 8-9; model answer หน้า 19; ข้อสมมติหน้า 6



ข้อ	คำตอบพร้อมเหตุผล
S3.1	checkout อยู่ที่ **`sprint_1_staging`**; HEAD ชี้ branch นี้และระบุ G
S3.2	G เพราะ local sprint อยู่ที่ G แต่ **`origin/sprint_1_staging`** อยู่ที่ F ส่วน F เป็น ancestor ของ G การมี commit G ใน local จึงยังไม่แสดงว่า remote-tracking ref นี้ตามมาถึง G แล้ว
S3.3	G เพราะ working directory สะอาดและ HEAD ระบุ G ผ่าน current branch
S3.4	fast-forward ไม่ได้ เพราะ E และ G แยกสายจาก common ancestor C และไม่มีปลายใดเป็น ancestor ของอีกปลาย ต้องทำ three-way merge; หากสำเร็จจะสร้าง merge commit ที่มี parents G และ E
S3.5	พิสูจน์ไม่ได้ เพราะ graph แสดง ancestry ไม่ได้แสดงเนื้อหาบรรทัดที่แก้ การเกิด conflict ขึ้นกับการเปลี่ยนแปลงที่เข้ากันไม่ได้
S3.6	**`git fetch origin`** อาจรับ commits/objects ใหม่และอัปเดต remote-tracking refs ตาม refspec แต่ไม่ merge งาน ไม่เลื่อน local branches หรือ HEAD และไม่เปลี่ยน working directory ให้โดยอัตโนมัติ

ตัวชี้ใน snapshot ก่อนทำคำสั่งใด ๆ

Reference	Commit
main; origin/main	B
feature/load_scores; origin/feature/load_scores	E
sprint_1_staging; HEAD ผ่าน branch นี้	G
origin/sprint_1_staging	F

ตรวจตนเอง: แยก local branch ออกจาก origin/*; อ่าน parent/common ancestor ถูก; ไม่อ้างว่าแยกสายแล้วต้อง conflict; ไม่เปลี่ยนคำว่า fetch ให้หมายถึง pull/merge

ข้อจำกัดของ snapshot: origin/* บอกสถานะ remote ล่าสุดที่เครื่องรู้ ไม่ได้ยืนยัน GitHub ณ เวลาปัจจุบัน คำตอบข้อ S3.2 จึงอธิบายความต่างของ refs ที่แสดง ไม่เดาว่ามีใคร push หรือแก้ remote หลัง snapshot หรือไม่